

Lebanese International University
Department of Computer Science and Information Technology
CSCI390 — Web Programming

MelodyHub

A Modern Music Streaming Web Application

Project Report — Phase 2

Submitted by:	[Your Full Name(s)]
Student ID:	52220031
Group:	[Group name / members]
Course:	CSCI390 — Web Programming
Semester:	Spring 2025-2026
Instructor:	[Instructor name]
Submission date:	May 2026

1. Abstract

MelodyHub is a responsive, single-page music streaming web application developed for Phase 2 of the CSCI390 Web Programming course. Building on the static HTML/CSS/JavaScript site delivered in Phase 1, this phase migrates the entire project to a modern **ReactJS** architecture powered by the **Vite** build tool and **React Router** for client-side navigation.

The application presents a polished, music-discovery interface featuring an animated landing hero, a grid of trending artists, a genre-browsing experience covering ten musical styles, and an interactive contact form. The user interface emphasizes a premium, glassmorphic visual style with animated gradients, smooth transitions, and a fully responsive layout that adapts from desktop to mobile, including a collapsible hamburger navigation menu. The project demonstrates component-based design, controlled form handling, version control with Git/GitHub, and cloud deployment on Vercel.

2. Introduction & Objectives

Music streaming platforms are among the most widely used web applications today. MelodyHub models such a platform's front end to practice real-world web development. The objectives of this phase are to:

- Apply web design and development principles using ReactJS as a frontend framework.
- Understand and implement responsive web design and modern UI/UX practices.
- Demonstrate version control and deployment skills using Git, GitHub, and Vercel.
- Convert a multi-file static site into a maintainable, component-based single-page application.

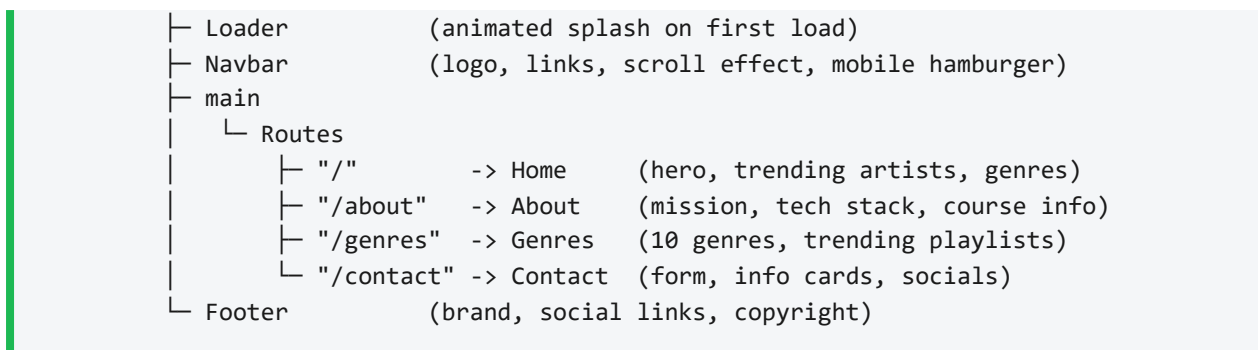
3. System Design

3.1 Architecture Overview

MelodyHub is a **Single-Page Application (SPA)**. A single HTML document (`index.html`) loads the React runtime, which mounts the application into a root DOM node. **React Router** then swaps page components in and out based on the URL — without full page reloads — giving a fast, app-like experience. Styling is handled by a single global stylesheet (`index.css`) ported and extended from Phase 1.

3.2 Component Hierarchy

```
main.jsx
├─ BrowserRouter
│   └─ App
```



3.3 Page Structure (4 Pages)

Route	Page	Description
/	Home	Landing hero with call-to-action, statistics, a grid of trending artists, and a genre preview.
/about	About	Project mission, design philosophy, technology stack badges, and course/group information.
/genres	Genres	Browsable grid of ten music genres plus curated trending playlists.
/contact	Contact	Controlled contact form with success feedback, contact info cards, and social links.

3.4 Responsive Design

The layout uses CSS Flexbox and Grid with relative units. Media queries at **768px** and **480px** reflow multi-column grids into single columns, and the desktop navigation collapses into an animated hamburger menu on smaller screens. The navbar is scroll-aware: its background becomes opaque after the user scrolls past 50 pixels.

4. Technologies Used

Technology	Role in the Project
React 18	Core UI library; builds the interface from reusable components.
Vite	Fast build tool and development server with hot-module reloading.
React Router v6	Client-side routing between the four pages.
JavaScript (ES6+)	Application logic, state, and event handling.
CSS3	Custom styling: gradients, glassmorphism (backdrop blur), animations, responsive media queries.
HTML5	Semantic markup and the SPA entry document.
Font Awesome 6	Vector icon set used throughout the UI.
Google Fonts (Inter)	Modern, legible typeface.
Git & GitHub	Version control and source code hosting.
Vercel	Cloud platform for continuous deployment and hosting.

5. Code Snippets (Key Parts)

5.1 Application Routing — App.jsx

React Router maps each URL to a page component, all wrapped in a shared layout:

```
export default function App() {
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    const t = setTimeout(() => setLoading(false), 1200)
    return () => clearTimeout(t)
  }, [])

  return (
    <>
      <Loader hidden={!loading} />
      <Navbar />
      <main>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/genres" element={<Genres />} />
        </Routes>
      </main>
    </>
  )
}
```

```

        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<Home />} />
      </Routes>
    </main>
    <Footer />
  </>
)
}

```

5.2 Scroll-Aware Navbar with Mobile Menu — Navbar.jsx

```

const [scrolled, setScrolled] = useState(false)
const [open, setOpen] = useState(false)

useEffect(() => {
  const onScroll = () => setScrolled(window.scrollY > 50)
  window.addEventListener('scroll', onScroll)
  return () => window.removeEventListener('scroll', onScroll)
}, [])

return (
  <nav className={`navbar${scrolled ? ' scrolled' : ''}`}>
    <ul className={`nav-links${open ? ' open' : ''}`}> ... </ul>
    <div className="hamburger" onClick={() => setOpen(o => !o)}> ... </div>
  </nav>
)

```

5.3 Controlled Contact Form — Contact.jsx

```

const [name, setName] = useState('')
const [submitted, setSubmitted] = useState(false)

const handleSubmit = (e) => {
  e.preventDefault()
  setSubmitted(true)
  setName(''); setEmail(''); setMessage('')
}

<form onSubmit={handleSubmit}>
  {submitted && <div className="form-success">Thanks! Your message has been sent.
</div>}
  <input value={name} onChange={(e) => setName(e.target.value)} required />
  <button type="submit">Send Message</button>
</form>

```

5.4 SPA Deployment Config — vercel.json

Rewrites all paths to the SPA entry so client-side routes work after a page refresh:

```

{ "rewrites": [{ "source": "/(.*)", "destination": "/" } ] }

```

6. Version Control & Deployment

The project is tracked with **Git**, with commit history hosted on **GitHub**. Deployment uses **Vercel**, which connects directly to the GitHub repository: every push triggers an automatic cloud build (`npm install + vite build`) and redeploys the live site. This means the production build runs on Vercel's servers, not the developer's machine.

- Vercel auto-detects the Vite framework — zero manual configuration.
- `vercel.json` ensures deep links such as `/genres` resolve correctly.
- Continuous deployment: pushing to the main branch redeploys automatically.

7. Screenshots of the UI

Replace each box below with a screenshot of the live deployed site. In Word: click the box, delete it, then Insert → Pictures. Capture each page on desktop, plus one mobile view.

[Screenshot 1 — Home page (hero + trending artists)]

[Screenshot 2 — About page (mission + tech stack)]

[Screenshot 3 — Genres page (genre grid)]

[Screenshot 4 — Contact page (form + info)]

[Screenshot 5 — Mobile view (hamburger menu open)]

8. Source Code Repository

GitHub repository (all code, assets, and documentation):

`https://github.com/[your-username]/melodyhub`

Live deployment (Vercel):

`https://melodyhub.vercel.app` (update with your actual URLs after pushing and deploying)

9. Group Contribution Statement

All team members contributed equally to the planning, implementation, and documentation of this project. Specific contributions:

Member	Contribution
[Member 1]	React component structure, routing, and Home page.
[Member 2]	Responsive CSS, animations, and About / Genres pages.
[Member 3]	Contact form, deployment, README, and this report.

Edit the table above to match your actual group members and their work.

10. Conclusion

MelodyHub Phase 2 successfully migrates a static website into a modern, responsive React single-page application. The project applies component-based architecture, client-side routing, controlled forms, and a responsive, visually polished UI, while practicing professional workflows through Git version control and automated cloud deployment. The result is a maintainable codebase that meets all functional and technical requirements of the assignment and provides a strong foundation for any future phases.